Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

on laboratory work no. 4

Course: Operating Systems
Topic: The xv6 Shell from the Outside

Done by: st. gr. FAF-242, Biriucov Alexandr
Checked by: Patricia Reițman

Chișinău, 2026

Purpose of the work

The purpose of this work is to study the shell by comparing the very small xv6 shell (user/sh.c, about 500 lines) with bash: to discover which features a shell must provide (running programs, redirection, pipes, sequences, background execution) and which are extras added by bash (globbing, variables, quoting, PATH search, history and completion). The work also includes adapting the sleep program to the renamed system call pause(), and writing a small program that prints its arguments, which shows exactly who expands * and who removes quotes: the shell, not the program or the kernel.

Part 1. Fix sleep for the current xv6

In August 2025 the xv6 authors renamed the system call sleep() to pause() (to avoid confusion with the kernel's internal sleep(chan, lock) function). A sleep program from Lab 2 that calls sleep() therefore no longer builds on the current tree.

1.1 Which name does the tree use?

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

Output:

```
vboxuser@ubuntu:~$ cd ~/xv6-riscv
vboxuser@ubuntu:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The grep result in user.h

1.2 The updated user/sleep.c

User.h declares pause, the call was changed as follows (the full program):

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
```

```
cat user/sleep.c
grep -n "_sleep" Makefile      # must show a line like:  $U/_sleep\
```

```
vboxuser@ubuntu:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
vboxuser@ubuntu:~/xv6-riscv$ grep -n "_sleep" Makefile
133:       $U/_sleep\
```

Figure 1.2 – The updated sleep.c and its UPROGS line

1.3 Build, boot and test

```
make qemu
$ sleep 20
$ sleep
(exit QEMU: Ctrl-A, then X)
```

sleep 20 waits 20 ticks and returns to the prompt; sleep without an argument prints "usage: sleep <ticks>".

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 1.3 – sleep running in xv6, with and without an argument

Part 2. bash versus the xv6 shell

2.1 Setup

```
mkdir ~/shtest && cd ~/shtest && touch README
```

Then every command of the table below is run twice: once in bash in this folder, and once at the xv6 $ prompt (make qemu). At the xv6 prompt type the commands one line at a time.

```
vboxuser@ubuntu:~/xv6-riscv$ cd ..
vboxuser@ubuntu:~$ mkdir ~/shtest && cd ~/shtest && touch README
vboxuser@ubuntu:~/shtest$ echo hello > f ; cat < f
hello
vboxuser@ubuntu:~/shtest$ ls | grep READ
README
vboxuser@ubuntu:~/shtest$ (echo a; echo b) | wc
      2       2       4
vboxuser@ubuntu:~/shtest$ echo one ; echo two
one
two
vboxuser@ubuntu:~/shtest$ echo bg &
[1] 5448
bg
[1]+  Done                    echo bg
vboxuser@ubuntu:~/shtest$ ls *
README  f
vboxuser@ubuntu:~/shtest$ echo $HOME
/home/vboxuser
vboxuser@ubuntu:~/shtest$ echo "a b"
a b
vboxuser@ubuntu:~/shtest$ mkdir d ; cd d ; ls ..
README  d  f
vboxuser@ubuntu:~/shtest/d$
```

Figure 2.1 – The commands run in bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README         2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d ; cd d ; ls ..
exec cd failed
.              1 1 1024
..             1 1 1024
README         2 2 2441
sleep          2 3 35280
cat            2 4 36728
echo           2 5 35592
forktest       2 6 18136
grep           2 7 44184
init           2 8 35976
kill           2 9 35512
ln             2 10 35304
ls             2 11 42880
mkdir          2 12 35568
rm             2 13 35552
sh             2 14 58312
stressfs       2 15 36416
usertests      2 16 206800
grind          2 17 51824
wc             2 18 37728
zombie         2 19 34848
logstress      2 20 37560
forphan        2 21 36352
dorphan        2 22 35800
sync           2 23 34944
```

```
console       3 24 0
f             2 25 6
d             1 26 32
$
```

Figure 2.2 – The same commands run in xv6

Things noticed

ls * prints an error, and echo $HOME prints $HOME itself.

- echo "a b" keeps the quote characters.
- After echo bg & the prompt may look odd. Just press Enter.
- On the last line, cd d fails with exec cd failed. That's expected.

2.2 Comparison table

Command	Tests	bash	xv6	Why
`echo hello > f ; cat < f`	redirection	hello	hello	Both shells implement > and < (xv6: REDIR) and ; (LIST).
`ls | grep READ`	pipe	README	README 2 <inum> <size>	Both have pipes. xv6 ls prints more per file: name, type, inode number, size.
`(echo a; echo b) | wc`	grouping, list	2 2 4	2 2 4	Both understand ( ) grouping; only the output format of wc differs.

Command	Tests	bash	xv6	Why
`echo one ; echo two`	sequence	one two	one two	The ; operator (LIST) runs the commands one after another.
`echo bg &`	background	[1] <PID> bg [1]+ Done ...	bg	Both run the command in the background (xv6: BACK). Only bash does job control and prints [1] and Done.
`ls *`	globbing	README	ls: cannot open *	bash expands * into file names. xv6 passes a literal * to ls, which tries to open a file named *.
`echo $HOME`	variables	/home/<user>	$HOME	xv6 sh has no variables; echo just prints the text it received.
`echo "a b"`	quotes	a b	"a b"	bash removes the quotes and keeps a b as one word. xv6 splits at the space and keeps the quote characters.
`mkdir d ; cd d ; ls ..`	PATH	d README	mkdir ok; "exec cd failed"; ls .. then lists the root directory	In xv6 cd works only when the line starts with cd (see Part 4), and ls is found only in the current directory (no PATH).
`arrow up, Tab, Ctrl+R`	history, completion	previous command, completion, history search	raw characters (^[[A), a tab character, nothing	The xv6 console only has Backspace, Ctrl-U (kill line), Ctrl-D (end of input) and Ctrl-P (process list).

Because cd d must be tested on its own in xv6, type these three lines separately:

```
$ mkdir d
$ cd d
$ ls
exec ls failed
$ /ls
$ /ls ..
```

```
$ ls
exec ls fa\iled
$ ls
exec \ls failed
$ /ls
.              1 26 32
..             1 1 1024
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
sleep          2 3 35280
cat            2 4 36728
echo           2 5 35592
forktest       2 6 18136
grep           2 7 44184
init           2 8 35976
kill           2 9 35512
ln             2 10 35304
ls             2 11 42880
mkdir          2 12 35568
rm             2 13 35552
sh             2 14 58312
stressfs       2 15 36416
usertests      2 16 206800
grind          2 17 51824
wc             2 18 37728
zombie         2 19 34848
logstress      2 20 37560
forphan        2 21 36352
dorphan        2 22 35800
sync           2 23 34944
console        3 24 0
f              2 25 6
d              1 26 32
$
```

Figure 2.3 – ls failing after cd d, and the working alternative

Answers to the Observe questions

1. Features of both shells and features only bash has. Both shells: run programs (fork + exec), input and output redirection (< and >), pipes (|), sequences of commands (;), background execution (&) and grouping with parentheses. Only bash: globbing (*, ?, [ ]), variables ($HOME) and command substitution, quoting (" " and ' '), the PATH search for commands, command history, arrow keys, Tab completion and Ctrl+R, job control (jobs, fg), && and ||, redirection of stderr (2>) and many built-ins.

2. Why does ls stop working after cd d in xv6, and how to run it anyway? xv6 has no PATH. When you type ls, the shell calls exec("ls") and the kernel looks for a file named ls relative to the current directory. All xv6 programs live in the root directory /, so after cd d the file d/ls does not exist and the shell prints "exec ls failed". The fix is to give the full path: /ls (or /ls .. to list the parent). Going back with cd .. also works.

3. Which missing feature would you miss most, and which is easiest to add? I miss command history/Tab completion and globbing most in daily work. The easiest to add is a simple PATH search: if exec(name) fails, try exec("/" + name) in runcmd (only a few lines of code). Variables or globbing would need a new expansion step in the parser.

Part 3. Who expands *? A program that prints its argv

3.1 The xv6 program args.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

The program was saved as user/args.c and added to UPROGS in the Makefile (line: $U/_args\). It simply prints how many arguments it received and each one between square brackets.

```
cd ~/xv6-riscv
cat user/args.c
grep -n "_args" Makefile
```

```
vboxuser@ubuntu:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
vboxuser@ubuntu:~/xv6-riscv$ grep -n "_args" Makefile
134:      $U/_args\
vboxuser@ubuntu:~/xv6-riscv$
```

Figure 3.1 – args.c and the UPROGS line

3.2 The Linux program args_linux.c

The same program with #include <stdio.h> instead of the two xv6 headers and return 0 instead of exit(0):

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

```
mkdir -p ~/os-lab4 && cd ~/os-lab4
# create args_linux.c here (nano args_linux.c), then:
gcc -o args args_linux.c
touch a.c b.c        # some .c files for the globbing test
```

3.3 Running the same lines in both shells

```
args one    two    three      # type several spaces between the words
args *.c "x y"
args $HOME
args 'single quoted'
```

```
vboxuser@ubuntu:~/os-lab4$ nano args_linux.c
vboxuser@ubuntu:~/os-lab4$ gcc -o args args_linux.c
vboxuser@ubuntu:~/os-lab4$ touch a.c b.c
vboxuser@ubuntu:~/os-lab4$ ./args one    two    three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
vboxuser@ubuntu:~/os-lab4$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [a.c]
argv[2] = [args_linux.c]
argv[3] = [b.c]
argv[4] = [x y]
vboxuser@ubuntu:~/os-lab4$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vboxuser]
vboxuser@ubuntu:~/os-lab4$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
vboxuser@ubuntu:~/os-lab4$
```

Figure 3.2 – args output in xv6

```
vboxuser@ubuntu:~/os-lab4$ ./args one    two    three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
vboxuser@ubuntu:~/os-lab4$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [a.c]
argv[2] = [args_linux.c]
argv[3] = [b.c]
argv[4] = [x y]
vboxuser@ubuntu:~/os-lab4$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vboxuser]
vboxuser@ubuntu:~/os-lab4$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
vboxuser@ubuntu:~/os-lab4$ ^C
vboxuser@ubuntu:~/os-lab4$
```

Figure 3.3 – args_linux output in bash

Expected results (compare with your output)

Line	bash (./args, with a.c b.c args_linux.c in the folder)	xv6 (args)
`one two three`	argc = 4: [./args] [one] [two] [three]	argc = 4: [args] [one] [two] [three]
`*.c "x y"`	argc = 5: [./args] [a.c] [args_linux.c] [b.c] [x y]	argc = 4: [args] [*.c] ["x] [y"]
`$HOME`	argc = 2: [./args] [/home/<user>]	argc = 2: [args] [$HOME]
`'single quoted'`	argc = 2: [./args] [single quoted]	argc = 3: [args] ['single] [quoted']

Answers to the Observe questions

1. Both outputs for args *.c "x y". Paste the two outputs from Figures 3.2 and 3.3. In bash the program received 5 arguments in my test folder (./args, the three .c files a.c, args_linux.c, b.c, and the single argument "x y"); in xv6 it received 4: args, the literal *.c, and the two pieces "x and y" (the quote characters are kept and the string is split at the space).

2. Who turns *.c into file names and removes the quotes? The shell. The program only receives the finished argv array and prints it. The kernel (exec) just copies that array to the new program without looking at it. bash expands *.c and removes the quotes before it starts the program; the xv6 shell does neither, so the program receives *.c and the quote characters exactly as typed (and in xv6 the root directory has no .c files anyway).

3. What happened to the extra spaces in one two three? Nothing is left of them: both shells use whitespace only as a separator between words, and any number of spaces counts as one separator. The program received exactly four arguments (the program name and three words) with no spaces inside them. To keep spaces in one argument in bash you must quote it, e.g. "one two".

Part 4. Read the shell

```
wc -l user/sh.c
grep -n "#define" user/sh.c      # the kinds of commands
grep -n "case " user/sh.c        # where each kind is run
grep -n "cd" user/sh.c | head
```

```
vboxuser@ubuntu:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
vboxuser@ubuntu:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

```
vboxuser@ubuntu:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

```
vboxuser@ubuntu:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
vboxuser@ubuntu:~/xv6-riscv$ grep -n -B2 -A6 "buf\[0\] == 'c'" user/sh.c
```

Figure 4.3 – The code that handles cd

```
grep -n "'\*'\|'\$'" user/sh.c      # finds nothing: no code for * or $
```

```
boxuser@ubuntu:~/xv6-riscv$ grep -n "'\*'\|'\$'" user/sh.c
vboxuser@ubuntu:~/xv6-riscv$
```

Figure 4.4 – Optional: searching sh.c for * and $ (no result)

Answers to the Observe questions

1. The five kinds of command in the xv6 shell. They are defined with #define in sh.c:

Kind	Character	Meaning
EXEC	plain words	a simple command: program name and its arguments
REDIR	< >	redirection of input or output to a file
PIPE	|	connects the output of one command to the input of another
LIST	;	run one command, then the next
BACK	&	run the command in the background (the shell does not wait)

Parentheses ( ) only group commands into a block; they do not create a sixth kind.

2. Where is cd handled and why is it not a separate program? cd is handled in main() of sh.c, in the loop that reads the input lines: if the line starts with "cd ", the shell calls chdir() itself and goes on to the next line, and only other lines are executed with fork() + runcmd(). The comment in the source says that chdir must be called by the parent, not the child. A separate program

would run in a child process and change only the child's own current directory, which disappears when the child exits; the shell would stay in the old directory. This is also the reason why cd in the middle of a line (mkdir d ; cd d) does not work in xv6: it is run in a child as a program called cd, which does not exist ("exec cd failed").

3. Code that handles * or $, and what it explains about Part 2. There is none: searching sh.c for * and $ finds nothing, because the xv6 shell has no globbing and no variables. This explains why ls * printed an error (the literal * was passed to ls), why echo $HOME printed $HOME and why args *.c in Part 3 showed [*.c]. The same reasoning applies to quotes, history and PATH: the xv6 shell is only about 500 lines because it provides just the five kinds of commands above, and everything else in bash is an addition.

Conclusions

In this laboratory work I updated my sleep program to the renamed system call pause() so that it builds on the current xv6, and compared bash with the xv6 shell. Both shells can run programs, redirect input and output, build pipes, run command sequences and background commands, and group commands with parentheses. bash additionally offers globbing, variables, quoting, a PATH search, history and Tab completion, and job control; the xv6 shell has none of them, which I confirmed by experiment (ls *, echo $HOME, echo "a b", ls after cd d).

With the args program I saw that the program never expands * or removes quotes: it only receives an array of strings from the shell through exec. In bash the shell prepares that array (globbing, variables, quotes) and in xv6 it only splits the line at spaces. Reading user/sh.c showed that the xv6 shell knows only five kinds of commands (EXEC, REDIR, PIPE, LIST, BACK) and that cd must be implemented inside the shell, because it changes the working directory of the shell process itself.